PYTHON PROGRAMMING | TRAINING MODULE

Python File Handling — The Complete Guide

Reading, writing, CSV, JSON, exceptions, os & pathlib — with practice sets

WHAT'S INSIDE

- 12 concept modules
- 40+ runnable code examples
- Cheat-sheet & quick reference
- 10 practice exercises
- Interview Q&A

Prepared for corporate & classroom training use

Table of Contents

01 What is File Handling & Why It Matters

File handling is how a Python program reads data from, and writes data to, files stored on disk (text files, CSVs, JSON, logs, images, and more). Without it, every program would lose all its data the moment it stopped running — variables live only in memory (RAM), and memory is wiped when the program exits. Files give a program **persistence**.

- **Persistence:** save results so they survive after the program closes.
- **Data exchange:** share data between programs, teams, or systems (e.g. exporting a report as CSV for Excel).
- **Configuration & logging:** read settings from config files, write logs for debugging and audit trails.
- **Large data processing:** work with data too large to hold in memory, one chunk at a time.

> **✓ TIP — Where you'll use this**
>
> Almost every real project touches files at some point — reading a sales export, writing an application log, generating a PDF invoice, or loading a JSON config for an API. File handling is a foundation skill, not a niche one.

Text Files vs Binary Files

Python treats every file as one of two kinds. Understanding the difference upfront avoids the single most common file-handling bug: opening a file in the wrong mode.

Type	Contains	Examples	Open with
Text file	Human-readable characters, decoded using an encoding (usually UTF-8)	.txt, .csv, .json, .py, .log	'r' / 'w' / 'a' (default text mode)
Binary file	Raw bytes — not meant to be decoded as text	.jpg, .png, .pdf, .mp3, .exe	'rb' / 'wb' / 'ab' (add the 'b')

02 Opening Files — open() and File Modes

Every file operation starts with the built-in `open()` function. It returns a **file object** that you then read from or write to.

```
file_object = open(file, mode='r', buffering=-1, encoding=None,
                   errors=None, newline=None)

# Most common usage:
f = open("sales_report.txt", "r", encoding="utf-8")
```

Syntax — the only two arguments you'll use 95% of the time are 'file' and 'mode'.

The Complete File Mode Table

Mode	Name	Behaviour
`'r'`	Read (default)	Opens for reading. Error if the file does not exist.
`'w'`	Write	Opens for writing. Creates the file if missing; ERASES existing content.
`'a'`	Append	Opens for writing at the end of the file. Creates the file if missing; keeps existing content.
`'x'`	Exclusive create	Creates a new file; raises FileExistsError if it already exists.
`'r+'`	Read & Write	File must exist. Pointer starts at position 0; can read and write.
`'w+'`	Write & Read	Erases existing content (or creates file), then allows read and write.
`'a+'`	Append & Read	Creates file if missing; writes go to the end; you can still read from anywhere.
`'rb'` / `'wb'` / `'ab'`	Binary versions	Same as above but data is read/written as bytes, not text.
`'t'`	Text mode (default)	Combine with above, e.g. 'rt' — usually omitted since it's the default.

■ WATCH OUT — Watch out — 'w' mode is destructive

Opening a file with mode `'w'` instantly truncates it to zero bytes, even before you write anything. If you only meant to add data, use `'a'` instead.

A Worked Example: Store Inventory File (Indore Branch)

```
# Creating and writing a fresh inventory file
f = open("indore_inventory.txt", "w", encoding="utf-8")
f.write("SKU101, Basmati Rice 5kg, 120\n")
f.write("SKU102, Sunflower Oil 1L, 340\n")
f.close()

# Reading it back
f = open("indore_inventory.txt", "r", encoding="utf-8")
print(f.read())
f.close()
```

03 Closing Files & the with Statement

Every file you open holds an operating-system resource (a file handle). If you forget to close it, you can leak resources, lose buffered writes that were never flushed to disk, or lock the file for other programs.

```
f = open("data.txt", "r")
data = f.read()
f.close()   # must always be called — easy to forget if an
            # error occurs above it
```

The manual way — works, but risky.

The Better Way: with (Context Manager)

The `with` statement automatically closes the file for you — even if an exception is raised inside the block. This is the professional, recommended way to handle files in Python.

```
with open("data.txt", "r", encoding="utf-8") as f:
    data = f.read()
    print(data)
# file is automatically closed here — no need to call f.close()

# Multiple files at once:
with open("input.txt", "r") as fin, open("output.txt", "w") as fout:
    for line in fin:
        fout.write(line.upper())
```

✓ TIP — Best practice

Always prefer `with open(...) as f:` over manual open/close. It's shorter, safer, and is what every real codebase and interviewer expects to see.

```
with open("data.txt") as f:
    pass
print(f.closed)   # True — confirms with() closed it automatically
```

04 Reading Files — read(), readline(), readlines()

Method	Returns	Best for
read()	Entire file content as one string	Small files you want to process in one go
read(n)	The next n characters/bytes as a string	Reading fixed-size chunks (large files)
readline()	The next single line (including \n)	Processing line-by-line with manual control
readlines()	A list of all lines in the file	When you need list operations (index, sort, etc.)
for line in f:	Iterates one line at a time (lazy)	Best for large files — memory efficient

```
with open("indore_inventory.txt", "r", encoding="utf-8") as f:
    whole = f.read()            # one big string
    print(whole)

with open("indore_inventory.txt", "r", encoding="utf-8") as f:
    first_line = f.readline()   # "SKU101, Basmati Rice 5kg, 120\n"

with open("indore_inventory.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()       # ['SKU101,...\n', 'SKU102,...\n']
    print(len(lines), "items in stock")

# The most memory-efficient way to read large files: iterate directly
with open("indore_inventory.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())     # .strip() removes the trailing newline
```

✓ TIP — Why 'for line in f' beats readlines() for big files

readlines() loads the ENTIRE file into memory as a list before you process anything. Iterating directly over the file object reads one line at a time, so a 2 GB log file never has to fit in RAM all at once.

05 Writing & Appending to Files

```
# write() — writes a string exactly as given (no automatic newline)
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Training session 1: File Handling\n")
    f.write("Training session 2: Exception Handling\n")

# writelines() — writes a list/iterable of strings (you add \n yourself)
lines = ["Module A\n", "Module B\n", "Module C\n"]
with open("notes.txt", "w", encoding="utf-8") as f:
    f.writelines(lines)

# append mode ('a') — adds to the end, never erases what's already there
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Module D (added later)\n")
```

■ WATCH OUT — write() vs print()

print() adds a newline automatically and writes to the console by default. f.write() does NOT add a newline — you must include \n yourself. (You can also do print(text, file=f) to redirect print's output into a file.)

06 File Pointer — seek() and tell()

Every open file keeps an internal cursor called the **file pointer**, marking where the next read/write will happen. `tell()` reports its current position; `seek()` moves it.

```
with open("indore_inventory.txt", "r", encoding="utf-8") as f:
    print(f.tell())        # 0  -> pointer starts at the beginning
    chunk = f.read(10)
    print(f.tell())        # 10 -> moved forward by 10 characters

    f.seek(0)               # rewind to the very start of the file
    print(f.readline())     # re-reads the first line

# seek(offset, whence)
# whence = 0 -> from start of file   (default)
# whence = 1 -> from current position
# whence = 2 -> from end of file  (binary mode only, in most cases)
with open("data.bin", "rb") as f:
    f.seek(0, 2)             # jump to end of file
    size = f.tell()          # this gives the file size in bytes
```

■ WATCH OUT — seek() with text files

In text mode, only seek(0) (start) and seek(offset from a value returned by tell()) are reliably portable. Arbitrary byte offsets and whence=1/2 are safest in binary ('b') mode.

07 Working with CSV Files

CSV (Comma-Separated Values) is the most common format for tabular data exchange — sales sheets, attendance logs, exported reports. Python's built-in `csv` module handles quoting, commas-inside-fields, and line endings correctly (which manual split(',') does not).

```
import csv

# --- Writing a CSV ---
rows = [
    ["Date", "City", "Product", "Units Sold", "Revenue"],
    ["2026-08-01", "Bhopal", "Basmati Rice 5kg", 42, 5040],
    ["2026-08-01", "Indore", "Sunflower Oil 1L", 65, 22100],
]
with open("sales_aug.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(rows)

# --- Reading a CSV ---
with open("sales_aug.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    header = next(reader)          # skip / capture header row
    for row in reader:
        print(row)                 # each row is a list of strings

# --- DictReader / DictWriter (work with column names, not positions) ---
with open("sales_aug.csv", "r", encoding="utf-8") as f:
    dict_reader = csv.DictReader(f)
    for row in dict_reader:
        print(row["City"], "->", row["Revenue"])

with open("summary.csv", "w", newline="", encoding="utf-8") as f:
    fieldnames = ["City", "Total Revenue"]
    dict_writer = csv.DictWriter(f, fieldnames=fieldnames)
    dict_writer.writeheader()
    dict_writer.writerow({"City": "Bhopal", "Total Revenue": 5040})
```

■ WATCH OUT — Always pass newline="" when writing CSV

On Windows, skipping newline="" causes csv.writer to insert a blank line after every row, because both Python's text-mode newline translation and the csv module add line endings. This one detail trips up almost every beginner.

08 Working with JSON Files

JSON (JavaScript Object Notation) is the standard format for structured/nested data — API responses, config files, app settings. Python's `json` module converts between JSON text and native Python dicts/lists.

Function	Direction	Use case
`json.dump(obj, f)`	Python object → JSON file	Save a dict/list directly to a file
`json.dumps(obj)`	Python object → JSON string	Get JSON as text (e.g. for an API response)
`json.load(f)`	JSON file → Python object	Load a config or data file into memory
`json.loads(text)`	JSON string → Python object	Parse JSON you already have as text

```
import json

store_config = {
    "store_id": "MP-BPL-01",
    "city": "Bhopal",
    "active": True,
    "categories": ["grocery", "dairy", "household"],
    "monthly_target": 850000
}

# Write to a JSON file (indent= makes it human-readable / "pretty")
with open("store_config.json", "w", encoding="utf-8") as f:
    json.dump(store_config, f, indent=4)

# Read it back
with open("store_config.json", "r", encoding="utf-8") as f:
    config = json.load(f)
    print(config["city"], config["monthly_target"])

# String <-> object without touching a file
text = json.dumps({"status": "ok"})     # '{"status": "ok"}'
obj = json.loads(text)                  # {'status': 'ok'}
```

09 Binary Files

Binary mode ('b') reads and writes raw bytes with no text decoding/encoding — required for images, PDFs, audio, or any format that isn't plain text.

```
# Copying an image file byte-for-byte
with open("logo.png", "rb") as source:
    data = source.read()

with open("logo_copy.png", "wb") as dest:
    dest.write(data)

# Reading a file in fixed-size chunks (efficient for very large binary files)
CHUNK = 4096
with open("training_video.mp4", "rb") as f:
    while True:
        chunk = f.read(CHUNK)
        if not chunk:
            break
        process(chunk)   # e.g. upload, hash, transform
```

■ WATCH OUT — Never open binary files without 'b'

Opening an image or PDF in plain 'r'/'w' mode forces Python to decode it as text using an encoding (like UTF-8), which will corrupt the bytes or raise a UnicodeDecodeError. If it isn't text, use 'rb'/'wb'.

10 Exception Handling in File Operations

File operations depend on things outside your program's control — a missing file, a full disk, a permissions issue. Production code always wraps file I/O in exception handling.

Exception	Raised when...
FileNotFoundError	You try to open a file that doesn't exist in 'r' mode
PermissionError	You lack OS permission to read/write/delete the file
IsADirectoryError	You try to open a directory path as if it were a file
FileExistsError	You use mode 'x' on a file that already exists
UnicodeDecodeError	The file's bytes can't be decoded with the given encoding
IOError / OSError	General input/output failure (disk full, device error, etc.)

```
try:
    with open("indore_inventory.txt", "r", encoding="utf-8") as f:
        data = f.read()
except FileNotFoundError:
    print("File not found — check the path and try again.")
except PermissionError:
    print("You don't have permission to read this file.")
except UnicodeDecodeError:
    print("Encoding mismatch — try encoding='latin-1' instead.")
else:
    print("File read successfully:", len(data), "characters")
finally:
    print("Attempted file read — operation complete.")
```

✓ TIP — else and finally with file I/O

The optional else: block runs only if no exception occurred — useful for 'success path' logic. finally: always runs, exception or not, making it the right place for cleanup that isn't already handled by with.

11 File & Directory Management — os and pathlib

Beyond reading/writing content, real programs need to check whether files exist, create folders, list directory contents, rename, delete, and move files. Two standard-library options: the older `os` / `os.path` module, and the modern, object-oriented `pathlib`.

Using the os module

```
import os

os.path.exists("sales_aug.csv")      # True / False
os.path.isfile("sales_aug.csv")      # True if it's a file
os.path.isdir("reports")             # True if it's a directory
os.path.getsize("sales_aug.csv")     # size in bytes
os.path.abspath("sales_aug.csv")     # full absolute path

os.mkdir("reports")                  # create one new folder
os.makedirs("reports/2026/august")   # create nested folders in one call

os.listdir(".")                      # list files/folders in current directory
os.rename("notes.txt", "training_notes.txt")
os.remove("old_file.txt")            # delete a file
os.rmdir("empty_folder")             # delete an EMPTY folder
```

Using pathlib (recommended for new code)

```
from pathlib import Path

p = Path("reports/2026/august/sales_aug.csv")

p.exists()          # True / False
p.is_file()          # True
p.parent             # Path('reports/2026/august')
p.name                # 'sales_aug.csv'
p.suffix              # '.csv'
p.stem                # 'sales_aug'

p.parent.mkdir(parents=True, exist_ok=True)   # create folders safely
p.write_text("Date,City,Revenue\n", encoding="utf-8")    # write, no open()
content = p.read_text(encoding="utf-8")                  # read without open()

for file in Path("reports").glob("*.csv"):     # find all CSVs in a folder
    print(file)
```

✓ TIP — os vs pathlib

pathlib is the modern, recommended approach — it's more readable (Path objects use / for joining, e.g. Path('reports') / 'august' / 'file.csv') and works identically on Windows, macOS and Linux. os/os.path still appears constantly in existing codebases, so recognise both.

shutil — copying and moving files

```
import shutil

shutil.copy("sales_aug.csv", "backup/sales_aug.csv")   # copy a file
shutil.move("sales_aug.csv", "archive/sales_aug.csv")  # move / rename
shutil.copytree("reports", "reports_backup")   # copy an entire folder
shutil.rmtree("old_reports")                   # delete folder + contents
```

12 Real-World Mini Project — Retail Sales Report

This end-to-end example combines everything covered so far: reading a CSV, handling missing files safely, aggregating data, and writing both a JSON summary and a plain-text report — a realistic task for a retail chain with stores across Bhopal, Indore and Jabalpur.

```
import csv
import json
from pathlib import Path
from collections import defaultdict

SOURCE = Path("sales_aug.csv")
REPORT_DIR = Path("reports")
REPORT_DIR.mkdir(exist_ok=True)

city_totals = defaultdict(float)

try:
    with open(SOURCE, "r", newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            city_totals[row["City"]] += float(row["Revenue"])
except FileNotFoundError:
    print(f"Source file {SOURCE} not found — check the path.")
else:
    # 1. Write a JSON summary
    summary_path = REPORT_DIR / "city_summary.json"
    with open(summary_path, "w", encoding="utf-8") as f:
        json.dump(city_totals, f, indent=4)

    # 2. Write a human-readable text report
    report_path = REPORT_DIR / "city_summary.txt"
    with open(report_path, "w", encoding="utf-8") as f:
        f.write("MONTHLY SALES SUMMARY BY CITY\n")
        f.write("=" * 32 + "\n")
        for city, total in sorted(city_totals.items(), key=lambda x: -x[1]):
            f.write(f"{city:<15}{total:>12,.2f}\n")

    print(f"Reports written to {REPORT_DIR.resolve()}")
finally:
    print("Sales report job finished.")
```

✓ TIP — What this example demonstrates

with + csv.DictReader for safe reading, try/except/else/finally for robust error handling, pathlib for cross-platform paths, and two different output formats (JSON for machines, .txt for humans) written from the same in-memory data.

13 Best Practices Checklist

- Always use `with open(...) as f:` instead of manual open()/close().
- Always specify `encoding='utf-8'` explicitly for text files — don't rely on the OS default.
- Use `'a'` mode deliberately — remember 'w' erases existing content instantly.
- Wrap file I/O in try/except and handle FileNotFoundError / PermissionError specifically.
- Prefer iterating (`for line in f`) over readlines() for large files.
- Use the csv and json modules instead of manual string splitting/formatting.
- Pass `newline=""` when writing CSV files to avoid blank-line issues on Windows.
- Prefer pathlib.Path over raw string paths for new code — it's safer and more readable.
- Never hardcode absolute paths from your own machine — use relative paths or Path objects.
- Close/verify files matter for large jobs: check disk space and permissions before big write operations.

14 Cheat Sheet — Quick Reference

Task	Code
Open & read whole file	`with open(f) as fh: text = fh.read()`
Read line by line	`for line in open(f): ...`
Write (overwrite)	`with open(f, "w") as fh: fh.write(text)`
Append	`with open(f, "a") as fh: fh.write(text)`
Read CSV as dicts	`csv.DictReader(fh)`
Write CSV	`csv.writer(fh).writerows(rows)`
Load JSON file	`json.load(fh)`
Save JSON file	`json.dump(obj, fh, indent=4)`
Check file exists	`Path(f).exists()`
Create folder(s)	`Path(f).mkdir(parents=True, exist_ok=True)`
Copy a file	`shutil.copy(src, dst)`
Delete a file	`os.remove(f) / Path(f).unlink()`

15 Practice Exercises

1	**Beginner** — Write a program that creates a file greeting.txt and writes "Hello, " to it, then reads and prints the content back.
2	**Beginner** — Write a program that counts the number of lines, words, and characters in any given text file.
3	**Beginner** — Given a file containing one name per line, write a program that prints all names in uppercase, one per line, without modifying the original file.
4	**Intermediate** — Write a program that appends a new timestamped log line to app.log every time it runs, without erasing previous entries.
5	**Intermediate** — Write a program that reads a CSV of student marks (Name, Subject, Score) and writes a new CSV containing only students who scored above 60.
6	**Intermediate** — Write a program that reads a JSON file of employee records and prints the names of employees whose department is "Sales".
7	**Intermediate** — Write a program that safely opens a user-specified file, handling FileNotFoundError and PermissionError with clear messages instead of crashing.
8	**Advanced** — Write a program that merges three separate CSV sales files (one per city) into a single combined CSV with an added 'City' column.
9	**Advanced** — Write a program using pathlib that scans a folder recursively and prints the total size (in MB) of all .csv files inside it.
10	**Advanced** — Write a program that reads a large text file in fixed-size chunks (not all at once) and counts how many times a given word appears, without loading the whole file into memory.

✓ TIP — How to use these

Work top to bottom — each level builds on skills from the one before it. For the Advanced set, time yourself: aim to finish each in under 15 minutes once you're comfortable with the module.

16 Interview Questions & Answers

Q. What is the difference between 'w' and 'a' file modes?
A. 'w' opens a file for writing and immediately erases any existing content (or creates a new file). 'a' opens for writing at the end of the file, preserving existing content.

Q. Why is the with statement preferred over open()/close()?
A. with uses a context manager to guarantee the file is closed automatically, even if an exception occurs inside the block — it's shorter and safer than manual close() calls.

Q. What's the difference between read(), readline(), and readlines()?
A. read() returns the entire file as one string; readline() returns just the next single line; readlines() returns a list of every line in the file.

Q. How do you read a very large file without running out of memory?
A. Iterate over the file object directly (for line in f) or read fixed-size chunks with read(n) inside a loop, instead of using read() or readlines() which load everything at once.

Q. What exception is raised when you try to open a non-existent file in read mode?
A. FileNotFoundError.

Q. What does seek(0) do?
A. It moves the file pointer back to the very beginning of the file, so the next read starts from position 0 again.

Q. Why must binary files be opened with a 'b' mode flag?
A. Without 'b', Python tries to decode the bytes as text using an encoding, which corrupts non-text data (images, PDFs, audio) or raises a UnicodeDecodeError.

Q. Why should you pass newline="" when writing a CSV file?
A. To prevent the csv module and Python's text-mode newline translation from both adding line endings, which otherwise produces an extra blank line after every row (mainly on Windows).

Q. What is the difference between json.dump() and json.dumps()?
A. json.dump() writes a Python object directly to an open file. json.dumps() converts it to a JSON string in memory, which you can print, send over a network, or write yourself.

Q. What is the advantage of pathlib over os.path?
A. pathlib represents paths as objects with methods and operators (e.g. the / operator to join paths), making code more readable and consistently cross-platform, versus os.path's string-based functions.

End of guide. Keep this alongside the Python fundamentals module for reference during lab exercises.